Solving PDEs with Backward Implicit Time Steps

Matthieu Gomez*

July 2026

The Julia package EconPDEs.jl solves the systems of nonlinear PDEs, possibly coupled with algebraic equations, that arise in continuous-time economic models: Hamilton–Jacobi–Bellman equations, asset-pricing equations, and systems that combine them. The method builds on the implicit finite-difference scheme of Achdou et al. (2022), but solves each implicit step fully — as a linear system when the valuation equation is linear, and as a nonlinear system when policies or equilibrium objects depend on the unknown solution — and adapts the step size along the way.

The object is a stationary system

$$0 = f\left(x, V, \partial_x V, \partial_{xx} V\right), \tag{PDE}$$

where V collects one or several unknown functions of the state x: value functions, price–dividend ratios, or wealth shares. Some rows of f may involve no derivative at all, such as market-clearing conditions or first-order conditions carried as separate unknowns. Other rows are linear in the derivatives once policies and prices are fixed, but nonlinear as equations in V because those policies and prices determine the drift, volatility, flow payoff, or discount rate. Thus a new candidate V does not merely re-evaluate a fixed equation: it changes the coefficients of the discretized equation itself.

At a high level, the algorithm has three steps:

1. Use upwind finite differences to turn the PDE into a finite system, $F(V) = 0$.
2. Embed the stationary equation in a false transient and take backward implicit steps: given the current iterate V_n, solve $(V_{n+1} - V_n)/\Delta = F(V_{n+1})$ for V_{n+1}. In a linear valuation problem each step is a single linear solve; when policies or equilibrium objects depend on V_{n+1}, the package solves the step with a nonlinear solver, Newton by default.
3. Adapt Δ: grow it when the stationary residual falls, shrink it when the residual rises, and retry failed steps with a much smaller Δ.

For readers used to Achdou et al. (2022), the differences are in steps 2 and 3. Their HJB update is semi-implicit: it computes policies from the old value function V_n, freezes them, and solves the

*I thank Valentin Haddad, Ben Moll, and Dejanir Silva for useful discussions.

resulting linear system implicitly in the new value function. EconPDEs instead solves the fully implicit nonlinear step to convergence when the step is nonlinear, so policies and equilibrium objects are updated as the candidate solution changes and are consistent with the new solution at convergence. It also treats Δ as a continuation parameter rather than a hand-chosen constant. Usually robustness and speed trade off: cautious iterations are stable but slow, while Newton steps are fast but local. Here the continuation combines the two. With Newton as the default nonlinear solver, the algorithm is robust when the guess is poor and much faster than a semi-implicit iteration near the solution, where it approaches quadratic rather than linear convergence.

Sections 1–3 follow these three steps in order. Section 1 explains why upwinding gives a Markov chain. Section 2 explains what a backward implicit false-transient step solves, how Newton solves the step when it is nonlinear, and how this differs from the semi-implicit update in Achdou et al. (2022). Section 3 explains why adapting Δ moves the algorithm from value function iteration toward policy iteration in pure HJBs and toward less anchored stationary-equilibrium updates in equilibrium systems. Section 4 discusses practical performance.

1 Discretization: the PDE becomes a Markov chain

The main message of this section is that, discretized with upwind finite differences, (PDE) is itself the valuation equation of a well-posed economic model — one in which the state moves on a finite Markov chain — and that this reading, more than any property of the calculus, is what delivers stability.

To fix ideas, take one state variable. The linear valuation equation is

$$\rho V(x) = u(x) + \mu(x)\,\partial_x V(x) + \frac{\sigma(x)^2}{2}\,\partial_{xx} V(x). \tag{1.1}$$

It prices a flow u discounted at rate ρ when the state follows a diffusion with drift μ and volatility σ. Discretize x on a grid with spacing Δx. The second derivative uses the centered difference. The first derivative is *upwinded*: approximated by the forward difference $(V_{i+1} - V_i)/\Delta x$ where the drift is positive and by the backward difference where it is negative. Writing $\mu^+ = \max(\mu, 0)$ and $\mu^- = \max(-\mu, 0)$, the equation at an interior grid point i becomes

$$\rho V_i = u_i + \lambda_i^+ (V_{i+1} - V_i) + \lambda_i^- (V_{i-1} - V_i), \qquad \lambda_i^\pm = \frac{\mu_i^\pm}{\Delta x} + \frac{\sigma_i^2}{2\Delta x^2}. \tag{1.2}$$

Equation (1.2) is exactly the valuation equation of a continuous-time Markov chain on the grid, which jumps one node up with intensity λ_i^+ and one node down with intensity λ_i^-: the drift becomes a directed jump intensity, the variance a symmetric one. Both intensities are nonnegative precisely because each first derivative is taken from the side the drift points to. Taking it from the other side subtracts $|\mu_i|/\Delta x$ from one of the λ's instead, and a negative λ gives the chain a negative

intensity — a negative transition probability, once time is discretized: the maximum principle fails, and iterative schemes built on the discretization explode. Upwinding is often glossed as "helping with convergence"; the sharper statement is that it is what makes the discretized problem an economy at all.

Stacking grid points, (1.2) reads

$$0 = F(V) = u + AV - \rho V, \tag{1.3}$$

where A is the intensity matrix of the chain: nonnegative off the diagonal, with rows summing to zero. At the edges of the grid the chain has no intensity pointing off the grid, so probability cannot leave: this is the default reflecting boundary. State-constraint boundaries follow the same logic, picking the one-sided derivative under which the drift points inward so that the chain never crosses the constraint. In the nonlinear case, the flow and the law of motion depend on the unknown solution, so (1.3) holds with $u = u(V)$ and $A = A(V)$.

This construction — approximate the continuous problem by an exact discrete one, rather than approximating the calculus — is the Markov chain approximation method of Kushner and Dupuis (2001). It also has a PDE-theoretic name: a scheme whose neighbor weights are nonnegative is *monotone*, and under the usual consistency, stability, and comparison-principle assumptions, monotone schemes converge to the unique viscosity solution as the grid is refined (Barles and Souganidis, 1991).[1]

2 Backward implicit steps and nonlinear solvers

The second step has two layers. The backward implicit step is the equation to be solved. The nonlinear solver is the algorithm used to solve that equation when the equation is nonlinear. This distinction matters because a linear valuation problem needs only one linear solve per step, while HJBs and equilibrium systems require an inner nonlinear solve.

To find a stationary solution, the algorithm embeds (1.3) in an artificial time dimension,

$$\partial_t V = F(V), \tag{2.1}$$

and marches toward the steady state $F(V) = 0$.[2] In fluid dynamics this device is called a *false transient*: the time path is not the object of interest, but following it is a robust way to reach the steady state.

[1] With several state variables, cross derivatives $\partial_{xy} V$ can push a scheme out of monotonicity when the local correlation is large relative to the grid. The package exposes directional cross-derivative stencils for this case and ships a diagnostic (`check_monotonicity`) that inspects the assembled Jacobian for wrong-signed neighbor weights.

[2] For Bellman equations the artificial time is time-to-go: marching (2.1) from a guess V_0 computes the value of a finite-horizon problem with terminal payoff V_0 and a growing horizon. In the package interface, the user returns the time derivative under the opposite sign convention, $\texttt{vt} = -F(V)$, i.e., the derivative with respect to calendar time.

In PDE language, the stationary equation is elliptic, often degenerate elliptic.[3] Adding the false time derivative turns it into a parabolic problem: an evolution equation in an artificial time variable, with the heat equation and time-dependent diffusion equations as canonical examples. Discretizing such a parabolic problem on a fine spatial grid produces a stiff ODE system, and the standard numerical response is backward Euler rather than a forward step whose stable size shrinks with the grid.

The design question is how to take the time steps in the false transient. From here on, subscripts on V index these time steps; grid points enter as arguments, as in $V_n(x_i)$.

The explicit step $V_{n+1} = V_n + \Delta F(V_n)$ is the familiar one: in numerical analysis it is forward Euler, and in the chain reading it is one round of value function iteration on a discrete-time version of the model with period length Δ, in which the intensities become transition probabilities $\Delta\lambda$. For these probabilities to be admissible the step must satisfy $\Delta \leq 1/\max_i(\lambda_i^+ + \lambda_i^-) \approx \Delta x^2/\sigma^2$, the CFL condition, and this cap is punishing on fine grids. With 1,000 grid points on a unit interval and $\sigma = 0.2$, it forces $\Delta \lesssim 2.5 \times 10^{-5}$; since each step then shrinks the error by a factor of only $1 - \rho\Delta$, halving the error with $\rho = 0.05$ takes on the order of half a million iterations. This is what *stiffness* means in practice: the grid, not the economics, dictates the pace of the iteration.

The package instead takes *backward implicit* steps, or backward Euler steps, solving

$$\frac{V_{n+1} - V_n}{\Delta} = F(V_{n+1}) \tag{2.2}$$

for V_{n+1}.

Case 1: Linear valuation problem. To see why this step is well behaved in the linear benchmark, take the law of motion, payoff, and discount rate as given while solving for V. Equivalently, in (1.3), take u and A as given. The implicit step is then a linear system, not a nonlinear problem. It solves to $V_{n+1} = [(1+\rho\Delta)I - \Delta A]^{-1}(\Delta u + V_n)$, and this operator has an exact probabilistic meaning:

$$V_{n+1}(x_i) = \mathbb{E}_{x_i}\left[\int_0^T e^{-\rho t}\, u(X_t)\, dt + e^{-\rho T}\, V_n(X_T)\right], \qquad T \sim \text{Exponential}(1/\Delta), \tag{2.3}$$

where X is the Markov chain of Section 1 started at x_i and T is an independent random horizon. In words: an implicit step of size Δ solves the model *exactly* out to a random horizon with mean Δ, and hands over to the previous iterate as continuation value at that horizon. Where an explicit step propagates the guess by one short period, an implicit step propagates it over a whole exponential lifetime.

[3] Elliptic means that the equation pins down a function of the state without a time direction, as in a Poisson equation. Degenerate elliptic allows the diffusion matrix to be singular, so the second-order part may vanish in some directions; HJB equations typically fall in this class.

Two properties follow. First, the step is stable for any Δ: the matrix $(1+\rho\Delta)I - \Delta A$ has a positive diagonal, nonpositive off-diagonal entries, and strict diagonal dominance — an *M-matrix* — so its inverse is nonnegative; it is the discounted expectation operator that (2.3) describes. Second, the step is a contraction: subtracting the fixed point from both sides of (2.3), the error shrinks in the sup norm by the average discount factor across horizons,

$$\mathbb{E}\left[e^{-\rho T}\right] = \frac{1}{1+\rho\Delta},$$

for every $\Delta > 0$. This is Blackwell's argument for the convergence of value function iteration, with $1/(1+\rho\Delta)$ in the role of the discount factor β — except that here the period length is a free parameter. A longer period means heavier discounting of the guess and faster convergence; as $\Delta \to \infty$ the guess drops out entirely and a single step solves the linear model outright. Nothing comparable is available for explicit steps, where raising Δ past the CFL cap does not speed convergence up but destroys it.

In mathematical finance, the same explicit–implicit distinction appears as trinomial-tree versus implicit finite-difference pricing. The structural difference between the two settings is the clock: an option has a known payoff at a known date, so the backward march is in real time and the step is tied to calendar accuracy. A stationary economic model has no terminal date, the transient is false, and nothing pins the step down.

Solving nonlinear steps. When F depends on the new candidate value through policies, prices, or equilibrium objects, (2.2) is a nonlinear equation. EconPDEs can use different nonlinear solvers; the default is Newton's method, with a trust-region variant available for harder systems. Write W for the unknown of the step while it is being solved; the accepted W becomes V_{n+1}. At fixed V_n, the residual of one implicit step is

$$R_\Delta(W) = \frac{W - V_n}{\Delta} - F(W).$$

Given an inner iterate W_k, Newton computes the residual and Jacobian, solves

$$DR_\Delta(W_k)\,\delta_k = -R_\Delta(W_k),$$

and updates $W_{k+1} = W_k + \delta_k$, damping the step with a line search when a full step overshoots. The inner loop stops when the residual of the implicit step is small. The outer loop, discussed in Section 3, then decides whether to accept the step and how to change Δ.

Case 2: Pure HJBs. A pure HJB adds a maximization to the linear valuation problem. Once a policy c is fixed, the equation is again a linear valuation equation; the nonlinearity comes from the fact that the optimal policy itself depends on the candidate value. Let $A(c)$ denote the finite-

difference operator obtained after fixing policy c. Given the previous outer iterate V_n, the fully implicit step asks for a new value W satisfying

$$\frac{W - V_n}{\Delta} = \max_c \{u(c) + A(c)\, W - \rho W\}.$$

This is a nonlinear problem in W because a new candidate value changes the marginal value and therefore the optimal policy c.

For this pure HJB, Newton has a familiar interpretation. At Newton iterate W_k, compute the policy c_k that is optimal for W_k. By the envelope theorem, the terms involving the derivative of c_k with respect to W_k drop from the Jacobian. The Newton step therefore holds the policy fixed and solves the linear policy-evaluation problem

$$\frac{W_{k+1} - V_n}{\Delta} = u(c_k) + A(c_k)\, W_{k+1} - \rho W_{k+1}.$$

Then the policy is recomputed at W_{k+1}, and the process repeats until the implicit nonlinear equation is solved. This is the policy-iteration logic inside the implicit step. The comparison to Achdou et al. (2022) is then transparent. Their update is semi-implicit for the nonlinear HJB: it chooses the policy using the old value V_n, calls it c_n, freezes it, and solves one linear system:

$$\frac{V_{n+1} - V_n}{\Delta} = u(c_n) + A(c_n)\, V_{n+1} - \rho V_{n+1}.$$

This is policy evaluation under the policy that was optimal for the old value function. Equivalently, it is the first frozen-policy Newton step on the fully implicit nonlinear equation, started from $W_0 = V_n$. EconPDEs keeps iterating Newton, so the policy is optimal for the new value function at convergence, not just for the old guess.

In discrete-time dynamic-programming terms, the semi-implicit update is one step of modified policy iteration: choose the policy from V_n, then partially or fully evaluate it. Small Δ looks like value function iteration; intermediate Δ is modified policy iteration; and $\Delta = \infty$ is Howard policy iteration (Puterman and Shin, 1978).

The semi-implicit update is a strong baseline for pure HJBs. EconPDEs can be more forgiving with poor guesses or aggressive Δ because it reoptimizes inside the implicit step and accepts a large step only after the policy is consistent with the new value. Still, the larger robustness gain appears in Case 3, where freezing equilibrium objects is no longer policy iteration.

Case 3: Equilibrium systems. In asset-pricing and general-equilibrium models, the residual includes prices, risk premia, volatilities, market-clearing equations, and sometimes algebraic constraints. These objects are not maximized-away controls, so the envelope theorem does not remove their derivatives from the Jacobian. A useful way to organize the computation is to keep the value functions and price functions as separate blocks and alternate between them.

There are two equivalent economic ways to write such a model, but they can be very different numerically. If the equilibrium prices have simple closed forms, one can eliminate them and substitute, for example,

$$r = R(V, \partial V), \qquad \kappa = K(V, \partial V),$$

directly into the HJB. This is often the cleanest formulation in one-state asset-pricing examples. In richer general-equilibrium models, however, the prices and risk premia can feed back into drift terms, volatility terms, upwind directions, and cross derivatives. Substitution then hides market clearing inside a highly nonlinear reduced-form PDE. An alternative is to carry the equilibrium objects as unknown functions and add their clearing equations as algebraic rows:

$$0 = H(V, r, \kappa), \qquad 0 = r - R(V, \partial V), \qquad 0 = \kappa - K(V, \partial V).$$

The system is larger, but it separates optimization from market clearing. This joint PDE/algebraic formulation is not a new economic equilibrium concept; it is a conditioning device. The algorithm alternates between the two blocks: holding prices fixed, solve the valuation equations for the value and claim-price functions; holding those valuation functions fixed, update prices from the market-clearing equations.

For more complex problems, especially higher-dimensional systems, repeating this alternating scheme until the values and prices stabilize is more robust than substituting prices away. See `garleanu_panageas_long_run_risk.jl` for an example with the interest rate and several prices of risk on the state grid.

Iterating Newton to convergence inside each step has different roles in Cases 2 and 3. In Case 2, it turns a single frozen-policy evaluation into a fully implicit HJB step for the current Δ. In Case 3, it is essential because the equations can remain nonlinear even after policy-like controls are frozen — recursive utility, equilibrium price–dividend ratios, wealth-share dynamics — so the semi-implicit update may have no linear system to fall back on.

3 Adapting the time step: pseudo-transient continuation

The remaining design choice is the schedule for Δ, and the answer is to let the residual choose it. Concretely, the outer loop is:

1. From the current iterate, attempt an implicit step (2.2) of size Δ (a linear solve in Case 1, a nonlinear solve in Cases 2 and 3).

2. If the inner solve converges, accept the step; grow Δ when the stationary residual $\|F\|$ falls, and shrink it when the residual rises.

3. If the inner solve fails, reject the step, divide Δ by 10, and retry.

4. Stop when $\|F\|$ is below tolerance.

The step size determines where the method sits between stabilized iteration and the stationary equations. As $\Delta \to 0$, an implicit step barely moves and the outer iteration behaves like value function iteration: globally stable but linearly convergent, at rate $1/(1+\rho\Delta)$. At $\Delta = \infty$, the anchor to the previous iterate vanishes and the step *is* the stationary equation. In a linear valuation problem, this is just the stationary linear system. In pure HJBs, the default Newton iterations then coincide with Howard policy iteration; the equivalence is classical (Puterman and Brumelle, 1979; Bokanowski et al., 2009). Policy iteration converges globally in textbook discounted problems, but that guarantee rests on every candidate value inducing a well-defined policy; in economic applications, a poor guess can imply negative marginal values, hence undefined or extreme policies, and the iteration breaks down. In equilibrium systems, the same limit removes the false-transient anchor from the stationary residual.

The adaptive rule moves across these regimes automatically. While the guess is poor, steps at large Δ fail, Δ stays small, and the algorithm behaves like value function iteration with a short period: slow, but hard to break. As the iterate approaches the solution, every step succeeds, Δ grows geometrically, and the last iterations are close to the stationary system, with the usual fast local convergence once policies and upwind directions stop switching. In pure HJBs this is the familiar movement from value function iteration toward policy iteration; in equilibrium systems it is the movement from a stabilized false transient toward the stationary equilibrium equations. In both cases, the rate of movement is set by observed progress rather than by the user.

This adaptation rule is the *pseudo-transient continuation* method of computational fluid dynamics, where steady flows are computed by marching false transients under exactly this kind of residual-based step control. Kelley and Keyes (1998) give formal conditions under which the scheme converges; Coffey et al. (2003) extend the analysis to systems in which some equations are algebraic rather than dynamic, the relevant case whenever the system couples PDEs with static conditions. The package handles these rows by dropping their $(V_{n+1} - V_n)/\Delta$ term (the `is_algebraic` flag).

4 Practical performance

In practice, the examples in the package solve quickly from simple initial guesses, using the same PDE formulation with only problem-specific solver options such as algebraic rows, bounds, or a smaller initial Δ. Two examples give the scale. The neoclassical growth model on a 1,000-point grid, started from the value of consuming gross output forever, converges in 11 outer iterations — a few hundredths of a second — with Δ growing from 1 to about 10^{12} along the way. The equilibrium model of Di Tella (2016), three unknown functions of two states with an algebraic market-clearing row (3,267 unknowns), converges from a flat guess in 14 outer iterations, under a second. This makes the solver useful not only for computing one equilibrium, but also for iterating

on parameter values, moment matching, and other outer loops where the model must be solved many times. The common formulation is also useful pedagogically: models that look different economically can be taught and compared as variations on the same discretized system.

In the nonlinear cases, the extra Newton work is cheaper than it may sound. The Jacobian is sparse because each equation touches only neighboring grid points. The package builds this sparsity pattern from the stencil, fills the matrix by finite differences — grouping columns by graph coloring so that a handful of residual evaluations recover all entries — and pays one sparse linear solve per Newton iteration even on fine grids. Recomputing the Jacobian is therefore a local, structured operation rather than a dense derivative calculation. The payoff is both robustness and speed: because each accepted step solves the current implicit equation more accurately than a single frozen-policy update, the adaptive rule can safely move to much larger Δ; the algorithm then takes far fewer outer steps than a fixed-step scheme and reaches the fast Newton regime sooner.

References

Achdou, Yves, Jiequn Han, Jean-Michel Lasry, Pierre-Louis Lions, and Benjamin Moll, "Income and Wealth Distribution in Macroeconomics: A Continuous-Time Approach," *Review of Economic Studies*, 2022, *89* (1), 45–86.

Barles, Guy and Panagiotis E Souganidis, "Convergence of approximation schemes for fully nonlinear second order equations," *Asymptotic analysis*, 1991, *4* (3), 271–283.

Bokanowski, Olivier, Stefania Maroso, and Hasnaa Zidani, "Some Convergence Results for Howard's Algorithm," *SIAM Journal on Numerical Analysis*, 2009, *47* (4), 3001–3026.

Coffey, Todd S, Carl T Kelley, and David E Keyes, "Pseudotransient Continuation and Differential-Algebraic Equations," *SIAM Journal on Scientific Computing*, 2003, *25* (2), 553–569.

Di Tella, Sebastian, "Uncertainty Shocks and Balance Sheet Recessions," *Journal of Political Economy*, 2016. Forthcoming.

Kelley, Carl Timothy and David E Keyes, "Convergence analysis of pseudo-transient continuation," *SIAM Journal on Numerical Analysis*, 1998, *35* (2), 508–523.

Kushner, Harold J and Paul Dupuis, *Numerical Methods for Stochastic Control Problems in Continuous Time*, 2nd ed., Springer, 2001.

Puterman, Martin L and Moon Chirl Shin, "Modified Policy Iteration Algorithms for Discounted Markov Decision Problems," *Management Science*, 1978, *24* (11), 1127–1137.

__ **and Shelby L Brumelle**, "On the Convergence of Policy Iteration in Stationary Dynamic Programming," *Mathematics of Operations Research*, 1979, *4* (1), 60–69.